DAT321/DIT843: Quality Assurance and Testing

Questions about the exam contact:
Examiner: Francisco Gomes, tel. 073 856 4439, francisco.gomes@cse.gu.se
Francisco will visit the exam halls around 15:00 and 16:30.

No aids are allowed for this exam.

The questions in this exam refer to the **ISO 25010:2011** that categorises four types of quality: process, internal, external and quality in use. The exam also covers the eight characteristics for product quality.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write correct, clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge or your learning of the topic.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product or software development process related to the questions. Avoid circular definitions and vague / generic answers. Use unique aspects of each topic covered in your answers.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper. The exam is anonymous, so do not leave any information that can reveal your identity.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 84: 4 (Pass with credit)
- 85 - 100: 5 (Pass with distinction)

The exam review is scheduled for:
Date: 2023-04-25 between 11:00 – 12:00.
Location: Jupiter room 400 (Jupiter building 4th floor – Software Engineering)

Question 1. [Total: 30 pts] Using the context below, answer the following questions:

You work for the Chat-Co, a company that offers a software product for chatting or messaging friends (think of WhatsApp, Facebook Messenger, etc.). Below we list important features and characteristics about the software product and its development process:

- **F1:** Each user account is connected to a single phone number. This number registered when creating the account.
- **F2:** User can chat with each other via direct messages or create group chats with many users.
- **F3:** Besides text messages, users can send audio or video clips (limit of 3 minutes per message), or image files.
- **F4:** Users can make video or audio calls with another user or a group chat that they are a part of. The user(s) on the other side of the call must accept the call before initiating the audio / video stream.
- **F5:** Users can choose to save their history of conversations in a server provided and maintained by the Chat-Co company.
- **F6:** Each message has three possible statuses shown to all users in the conversation: sent, received, and read. In a group message, those status only trigger when all participants in the group receive or read the message.
- **F7:** The ChatApp can be used in mobile phones, tablets, or personal computers (PCs).
- **F8:** The user interface must adapt, i.e., be responsive to the different layouts of screens from multiple types of devices.
- **F9:** The ChatApp can import a list of contacts from the user's personal accounts (Google, Apple, Facebook, etc.).
- **F10:** The development team does not have a fixed release calendar. New features and bug fixes are only added based on requests by the community of users.
- **F11:** Users can customize their user interface by choosing different color themes, backgrounds, fonts, and text color in their chats.

Note: For the questions below, you are allowed to add features or decisions about the software development process (team size, technologies, etc.). However, any assumption related to your answer: (i) must be clearly state, and (ii) cannot contradict any of the points above.

a. **[8 pts]** Choose **exactly two** software product quality attributes, **and, for each,** provide (i) a brief explanation describing the quality attribute, and (ii) why they are relevant to the software product described above. When justifying your choices, you **must** refer to one or more of the points (F1 - F11) listed above.

[+4 pts per attribute] You cannot write vague or circular definitions. Both the explanation and the justification should emphasise the unique aspect of the quality attribute. For specific definitions, check the lecture slides:

Usability: Ability for users to interact with the software product or complete tasks by using it. Relevant due to many aspects of the exchange of messages and media between different (groups of) end-users. Features related to the user interface, or any type of end-user feedback: F3, F4, F7, F8, 11

Maintainability: Degree to which we can change the software without additional cost, effort or disrupting its functional or non-functional aspects. Relevant due to the unplanned releases, so the software should accommodate and integrate modifications more easily and avoid technical debt. F10

Performance Efficiency: Degree to which the software requires and uses computational resources. Relevant due to the need to exchange information within some reasonable time. Some features are: F3, F4, F5

Portability: Degree to which the software runs on different hardware devices. Relevant since the specification says that the ChatApp should run on different devices such as laptops, phones or tablets (F7)

Compatibility: Degree to which the software can share and exchange information with other applications. Relevant as ChatApp needs to communicate with external applications or devices (e.g., camera app) to send different types of media. Features F3, F4, 5

Security: Degree to which the software can ensure correct access levels to the user's information. Relevant as messages and chats can reveal private information from the user and ChatApp requires some identification way to register its users (F1).

Reliability: Degree to which the software can function under specific conditions and during a specific period of time. Relevant as the messages are exchange between users both asynchronously (text chat) and synchronously (calls), so availability of these services is important to avoid crashes or poor usage of the software. Features F2 – F6.

b. [6 pts] For each of the quality attribute chosen above create a quality scenario that includes: (i) source, (ii) stimulus, (iii) artefact, (iv) environment, (v) response, and (vi) response measure.

[+3 pts per scenario] These will depend on your choice of attributes above. Some examples:

Usability:
1- Source: End-User (Sender)
2- Stimulus: Paste a video URL to the message field.
3- Artefact: Message Field.
4- Environment: The video is available online and the device is connected to the Internet.
5- Response: A thumbnail of the video is shown in the message view.
6- Response measure: Bigger (dimensions) in the message field to fit the thumbnail.

Performance Efficiency:
1- Source: End-User (Sender)
2- Stimulus: Write a message and press the send button.
3- Artefact: User's client interface.
4- Environment: The sender device is connected to the internet, but the receiver is not available.
5- Response: The message is shown in the chat history screen, but the message is tagged as non-received.
6- Response measure: Time to receive the visual feedback for non-received status.

c. **[10 pts]** Explain the difference between continuous integration and continuous deployment. Describe one continuous deployment strategy and give one example on how we can use that continuous deployment strategy to at least one of the listed features in the ChatApp.

[+4 pts] The difference is that: CI focuses on **automated build (compilation) and testing** by checking the dependencies of the different parts of the software product, but the deployment can be done manually; in contrast, CD requires an automated deployment which means automatically **installing the integrated product (binary, or package) into the production** environment so that the product is available to its end-users.

[+6 pts] There are different techniques, but two very popular ones are AB testing and canary releases. We can use AB testing to define details of the interface of the ChatApp application, such as layout of buttons, text fields, how to let users know the status of messages, etc (Features F6, F7, F8 or F11). Canary releases can be used to release updates in the features or try out different optimisation algorithms for, e.g., streaming calls (F2, F3, F4) and verify the impact of those new features before deciding to roll them out to all users.

d. **[6 pts]** Chat-Co wants to promote sustainability in the product that we are creating. Using the Sustainability Awareness Framework, you must (i) **choose or suggest two functionalities for the app** and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the **corresponding dimension of sustainability** in which those effects.

[+3 pts per correct suggestion] There are many possible options, the important criteria is to have a clear distinction between immediate, enabling and structural effects. Some examples:

Suggested feature: Introduce automated moderation of group chats to avoid cyber bullying.

- Immediate effect: Removing, deleting, or preventing harmful or harassing messages to be sent to groups of users. (individual)
- Enabling effect: Improved sense of community in groups, and creation of safe spaces (social).

- Structural effect: ChatApp becomes a preferred place for users that belong to minority groups since they can safely connect to other users. (social)

Suggested feature: Improve the algorithms and infrastructure needed to store history information (e.g., delete history)

- Immediate effect: With less demand or need to store history information, fewer storage resources are needed. (technical)
- Enabling effect: Cheaper infrastructure needed for the software system, such as servers or cloud platform providers (economical)
- Structural: Reduced demand of energy and other resources associated with the cheaper infrastructure. (environment)

Question 2. [Total: 35 pts] Based on your knowledge of Property-based testing (PBT) and Black-box Testing, answer the questions below. We listed also some examples of methods that can be used when writing your properties.

List of methods that can be used to write your properties:

`sum(int[]): int`	Returns the sum of the integers in the specified array.
`length(int[]): int`	Returns the number of elements in the array.
`sort(int[]): int[]`	Returns the array with its values sorted in an ascending order.
`reverse(int[]): int[]`	Returns the array where its values are in reversed positions.
`max(int[]): int`	Returns the highest integer value in the array.
`min(int[]): int`	Returns the lowest integer value in the array.

Specification:[1] The function `shift(int[] numbers, int direction)` **receives** two arguments: (i) an array of integers, and (ii) an integer value that can be negative, positive or zero. The function returns a new array that contains the same numbers as the input array but shifted depending on the value of the direction.

- If the direction is positive, the values are shifted one index to the right.
- If the direction is zero, the values are not shifted, and the array remains the same.
- If the direction is negative, the values are shifted one index to the left.

Note: Since the implementation is done in Java, there cannot be null elements inside the array of integers.

[1] The specification and test cases for this question were adapted from www.codingbat.com.

Below, we list some examples of test cases (inputs and expected outputs) for the SUT above:

T1	`shift([1, 2, 3, 4], -1) → [2, 3, 4, 1]`
T2	`shift ([5, 3, 2], 0) → [5, 3, 2]`
T3	`shift ([2, 1, 0, -1, -2], 1) → [-2, 2, 1, 0, -1]`
T4	`shift ([3, 2, 2], -5) → [2, 2, 3]`
T5	`shift ([2, 2, 2], 3) → [2, 2, 2]`
T6	`shift ([1], 1) → [1]`
T7	`shift ([], 0) → []`
T8	`shift (null, 1) → Exception:"Invalid array of integers."`

a. **[12 pts]** Create **at least four** properties for the SUT above. You can write the properties using natural language, mathematical notation, or pseudocode. Feel free to use any of the support functions for arrays of integers listed in the beginning of this question. Your properties will be evaluated based on the diversity of their input and output.

[+3 pts] per property, but the properties should cover different cases. Note: Remember that your properties should work for a set of input, rather than specific input values (otherwise, they become test cases, not properties).

Many of the functions above can be used in the properties, as the elements or the size of the array is not being changed by the function. So, some examples of properties are:

- For any integer x, and array a:
  - length(a) == length(shift(a,x))
  - sort(a) == sort(shift(a,x))

- For the below, you should specify any integer x, a non-empty array a:
  - sum(a) == sum(shift(a,x))
  - min(a) == min(shift(a,x))
  - max(a) == max(shift(a,x))

Some other examples that are more related to the input and the SUT are:

- For any array a:
  - a == shift(a, 0)

- For any integer x, and array a: (similar to a reverse pattern)
  - shift(a, x) == shift(shift(a,x), -x)

- For y = length(a):

- `a == [for i in 1..y: shift(a,1)], i.e., if you repeatedly give 'a' to the shift function 'y' times, we result with the same array as the input (should work for shifting either right or left).`

b. **[8 pts]** Classify each property above into two types: (1) a property created using one of the patterns for creating properties in PBT (which pattern was used), or (2) a property that is particular for the SUT specification. Justify why does your property falls into that category.

[+2 pts] per justification. This will depend on your property. For the ones regarding using length, sum, sort, all of that is SUT specific, as they only work because of the specification of the shift function (i.e., they would not work for other functions). From the example above, the only pattern found is the reverse (considering x and -x).

c. **[15 pts]** Considering the concept of the program derivative to identify boundary behavior in the **`int[] shift(int[] numbers, int direction)`** function above, answer the question. Your colleague Charlie suggests using the following distance functions:

- Input distance: Absolute difference between a pair of **direction (int)** arguments.
- Output distance: The sum of the elements in the resulting array.

Do you agree with Charlie's suggestions for the program derivative? **If yes**, justify your answer with examples of boundaries using those distances. **If no**, justify your answer and suggest other distance(s) function(s) to obtain the derivative.

[+3 pts] Explain whether the functions presented are applicable in the context of the program derivative.

The Program Derivative measures the sensitive of the program to change, and it relies on having inputs that are similar (low distance), and outputs that are dissimilar (high distance values).

[+8 pts] Find the problems in relation to the functions presented and boundary behavior: The chosen **input function is not suitable because** it does consider the input array, which is the main value that will affect the output. For instance, arrays of different length will have the same input distance, which would belong to different partition classes. Similarly, the **output distance is not suitable** because the sum of any output will be the same as the output, so the main differences between pairs of outputs, will be in the indices of the arrays.

[+4 pts] Present one example of a new distance function: For this, you could describe different functions, if they would consider important elements of the arrays **and** the direction parameter. If the partitions are based on the length and direction of the shift:

E.g., for TCa = (a, directionA), TCb = (b, direction):

```
- di(TCa, TCb) = |(length(a) * directionA) - length(b) * directionB)|
- do(TCa, TCb) = sum(OutputA[i] - OutputB[i]).
```

Note that if input is small (i.e., only direction change, then the output will cause a difference in the indices.). Some other options would be considering indices of the max / min elements for input or output. The goal of the question is to show how your understanding of how input and output distances should be defined in terms of input and output data (including all parameters that can affect the outcome).

Question 3. [Total: 35 pts] Using the code below answer the questions.

The function below parses an array of exam points (as integers) into the corresponding course grade using the same conversion scale described in this exam - see first page.

The program can throw two types of Exceptions: the first covers cases where the array of points is empty or null, whereas the second cover cases where there is any invalid point value in the input array (outside the range 0 and 100). Java does not allow null values inside the array.

```
01.  public static String[] gradeExams(int[] examPoints) throws Exception {
02.    if(examPoints == null || examPoints.length == 0) {
03.      throw new Exception("Error. Array of points cannot be empty or null.");
04.    }
05.    String[] courseGrades = new String[examPoints.length];
06.    for(int i = 0; i < examPoints.length; i++) {
07.      int currentPoints = examPoints[i];
08.      if(currentPoints >= 0 && currentPoints < 50){
09.        courseGrades[i] = "U";
10.      } else if(currentPoints >= 50 && currentPoints < 70){
11.        courseGrades[i] = "3";
12.      } else if(currentPoints >= 70 && currentPoints < 85){
13.        courseGrades[i] = "4";
14.      } else if(currentPoints >= 85 && currentPoints <= 100){
15.        courseGrades[i] = "5";
16.      } else {
17.        throw new Exception("Invalid. Points must be between 0 and 100.");
```

```
18.       }
19.     }
20.     return courseGrades;
21.   }
```

a. **[10 pts]** Explain the differences between statement, branch, and condition coverage criteria. Additionally, create at least one test suite for the `gradeExams` function that illustrate your reasoning.

[+6 pts] The difference is that statement covers a lines of code independently of triggering different paths in the behaviour, whereas branch counts the number of times each branch was evaluated to true **OR** false. Finally, branch and condition differ in cases a branching statement (ie., if-then) is **based on a Boolean expression** where each specification condition inside that expression must evaluate to both try true or false.

[+4pts] Some examples of test suites (sets of tests, are): Considering just the initial lines of code (also application for the if-statement in lines 06 – 17):

TS1:
TC1: []; Exp. Out: Exception
TC2: [10, 55, 70, 90]; Exp. Out: [U, 3, 4, 5]
TC3: [120]; Exp. Out: Exception

TS1 has 100% statement coverage, but not all branches are evaluated to true or false (e.g., the first branch is evaluated to both false and true, but not all individual conditions inside that branch.

b. **[10 pts]** Create at least four equivalent partition classes for the `gradeExams` function. Your partition classes will be evaluated based on the diverse behavior that they exercise.

[+2 pts per correct partition. +2 pts for clarity and organization] There can be one partition for each behavior in the if statement **in relation to the input array**. Define them using the set of input and corresponding triggered behavior.

Partition 1: Arrays that are either null or empty.
Equivalent behavior: The Exception for invalid array is thrown.

Partition 2: Arrays that contain the same grade for all points within the valid range.
Equivalent behavior: Output array will always have the same elements.
Note: This includes arrays of length 1.

Partition 3: Arrays that only contains with max 2 repeated grades.
Equivalent behavior: The resulting array will not include at least one of the grades.

Partition 4: Arrays that contain all and valid possible grades.
Equivalent behavior: The resulting array will have all three grades at least once.

Partition 5: Arrays with at least one point outside the range.
Equivalent behavior: Throw exception about points being out of range.

c. **[10 pts]** Using the partition classes that you defined above, create test cases that reveal at least three boundaries. You must indicate the corresponding equivalent partition classes of each boundary listed.

[+3 pts per correct partition. +1 pts for clarity and organization]
Boundary 1: P1 and P2

- TC1: []. Exp. Output: Exception
- TC2: [89]. Exp. Output: ["5"]

Boundary 2: P2 and P3

- TC1: [49, 70, 71, 84]. Exp: [U, 4, 4, 4]
- TC2: [49, 50, 70, 85]. Exp: [U, 3, 4, 5]

Boundary 2: P2 and P4

- TC1: [49, 100]. Exp: [U, 5]
- TC2: [49, 101]. Exp: "Exception"

d. **[5 pts]** For the code above, (i) create a mutant using the Conditional Operation Replacement as a mutation operator. You only need to write the specific line of code that you mutated. (ii) Show an example of a test case that would kill that mutant.

[2 pts] The conditional operational replacement (OCR) changes Boolean operators, so any line with the if statement would be suitable for mutation:

- Line 2: Change || to &&
- Lines 8, 10, 12, 14: Change && to ||

[3 pts] If we change line 8, we can kill this mutant by using the test:

- TC: [90]. Exp: [5], Actual due to mutant: [U]